Dataset Understanding, Quality and Preprocessing Report

Every figure below is computed by `ml/data_report.py` from the live data generators and the real preprocessing pipelines. Nothing is transcribed by hand.

1. Datasets at a glance

Dataset	Samples	Target	Classes	Balance (min/max)	Source
Onboarding documents	4,950	document type	9	1.00	Generated (`ml/synth.py`)
Employee messages	3,360	intent + sentiment	8 + 3	1.00	Generated (`ml/text_synth.py`)
Onboarding journeys	6,000	stalled (binary)	2	0.76	Simulated (`ml/risk_synth.py`)

Why the data is synthetic

Real onboarding documents are identity and financial records that cannot be collected or published, and real onboarding outcomes do not exist for a system that has not been deployed. Both corpora are therefore generated. The generators are built to make the task realistically hard rather than flattering:

- Documents are produced from 21 templates across 9 classes, then degraded in three tiers (clean=2,005, degraded=1,841, severe=1,104).
- Messages come from 45 templates and include deliberately

unresolvable cases, so some error is irreducible.

- Journeys are a latent-variable process (engagement, friction, support) with an

explicitly unobserved component, so no feature set can predict the outcome perfectly.

Limitation, stated plainly: these metrics measure internal consistency with a generator, not real-world capability. No real document, message or outcome has been observed.

2. Data understanding

2.1 Onboarding documents

- Samples: **4,950** across **9** classes
- Text volume: 1,723,396 characters / 240,834 words
- Document length: mean 348 chars (median 342, min 75, max 727, sd 121)
- Mean words per document: 49

Class	Count
background_check_consent	550
bank_details	550
education_certificate	550
govt_id	550
ip_agreement	550
offer_letter	550

Class	Count
other	550
professional_license	550
safety_certification	550

2.2 Employee messages

- Samples: **3,360**, 8 intents, 3 sentiment classes
- Mean message length: 73 chars

Intent	Count
document_help	420
escalation	420
it_access	420
kt_schedule	420
payroll_benefits	420
policy_question	420
smalltalk	420
status_check	420

Sentiment distribution: positive = 913, neutral = 1,318, negative = 1,129

2.3 Onboarding journeys (tabular)

- Samples: **6,000**, Features: **12**, Target: `stalled` (binary)
- Class distribution: stalled 2,599 (43.3%), not stalled 3,401
- All features are numeric (float64); department is used to shape the simulation

but is not exposed as a model input.

Feature	Min	Max	Mean	Median	SD	Corr. with target
days_since_start	3.00	60.00	31.43	31.00	16.71	-0.008
stage	0.00	5.00	1.80	2.00	1.22	-0.343
progress	0.00	100.00	36.11	33.36	23.52	-0.353
pct_tasks_complete	0.00	1.00	0.36	0.33	0.22	-0.377
days_since_last_activity	0.00	30.00	2.82	1.45	3.90	+0.286
docs_pending	0.00	9.00	2.13	1.00	2.28	+0.386
docs_rejected	0.00	5.00	0.29	0.00	0.61	+0.210
doc_resubmissions	0.00	7.00	0.63	0.00	0.94	+0.239
kt_completed_pct	0.60	100.00	31.66	27.59	20.42	-0.318
chat_messages	0.00	32.00	5.13	4.00	3.67	-0.065
avg_sentiment	-1.00	1.00	-0.03	-0.06	0.72	-0.446

Feature	Min	Max	Mean	Median	SD	Corr. with target
seniority	0.00	2.00	0.99	1.00	0.82	-0.035

Stall rate by department:

Department	Stall rate
Operations	53.0%
Legal	46.5%
Supply Chain	45.6%
Sales	39.8%
Engineering	37.7%
Design	37.5%

3. Data quality checks

Check	Documents	Messages	Journeys
Missing values	0	0	0
Exact duplicates	0	200	0
Duplicates after normalising case/whitespace	0	293	n/a
Empty records	0	0	0
Zero-variance features	n/a	n/a	0

Findings

Documents are clean. Zero exact duplicates and zero near-duplicates across 4,950 records, and zero overlap between the train and test folds. Although only 21 templates are used, every record randomises names, dates, identifiers and degradation, so no two documents come out identical.

Messages are not clean, and this is a real defect. 200 exact duplicates and 293 near-duplicates (8.72%) exist in 3,360 records, and **70 messages appear in both the training and test folds**, 8.3% of the test set. Short messages such as "thank you" and "any update?" are produced identically by different draws, so a stratified random split cannot separate them.

This is **train/test leakage**: the model is tested partly on strings it has already seen. The correct fix is to deduplicate before splitting, or to group identical strings into the same fold.

Measured impact of the leakage

The obvious hypothesis is that this leakage inflates the scores, and that it explains sentiment reaching 0.996. That hypothesis was tested rather than assumed. One training run, with the test set divided into rows that also appear in training (leaked) and rows that do not (clean). A positive gap would be memorisation:

Model	Overall	On leaked rows	On clean rows	Gap
Sentiment	0.9964	0.9868 (76 rows)	0.9974 (764 rows)	-0.0105
Intent	0.9369	0.9467 (75 rows)	0.9359 (765 rows)	+0.0107

The hypothesis was wrong. Sentiment scores *lower* on the leaked rows (0.9868) than on the clean ones (0.9974), and the intent gap is within noise. The leakage is real but its measured effect is negligible.

The explanation is that the duplicated strings are the *short, ambiguous* ones, "thank you", "any update?", which are both the most likely to be generated twice and the hardest to classify. Duplication is concentrated in the difficult cases, so it depresses the leaked-row score rather than inflating it.

Sentiment's 0.996 therefore still needs explaining, and leakage is not the explanation. It remains a **template-separability ceiling**: the generator marks sentiment with stock phrases that a linear model separates almost perfectly. That is a property of the data, not evidence the model understands sentiment.

Outliers (IQR rule, tabular dataset)

Feature	Outliers	% of rows
docs_rejected	1,390	23.17%
days_since_last_activity	477	7.95%
doc_resubmissions	295	4.92%
docs_pending	272	4.53%
chat_messages	206	3.43%
kt_completed_pct	43	0.72%

Outliers were **retained**, not removed. These are counts and durations whose long right tail is the signal of interest, an employee inactive for 25 days is exactly the case the model must catch, not noise to discard.

4. Preprocessing: what, why, result

Step	Why	Result
Lowercasing + accent stripping	OCR and typing produce inconsistent case	applied to all text features
Word 1-2 gram TF-IDF	captures topical vocabulary and short phrases	11,482 features
Character 3-5 gram TF-IDF	survives OCR damage that destroys whole words	unioned with word features
Sublinear TF	dampens the effect of a term repeated many times in one document	applied
`min_df=2` pruning	removes tokens appearing once, which cannot generalise	30,426 features dropped
Stratified split	preserves class proportions in both folds	see section 5
StandardScaler (tabular)	features span different units and ranges	zero mean, unit variance
Scaler fitted inside a Pipeline	prevents test statistics leaking into training	enforced by sklearn

4.1 Before vs after: text vectorisation (document corpus)

Measure	Before	After
Representation	4,950 raw strings	sparse matrix 4,950 x 11,482
Unique tokens	11,056	11,482 word n-gram features
Features before pruning	41,908	11,482 (min_df=2)
Density	dense text	99.26% sparse, 85 non-zeros per document

4.2 Before vs after: feature scaling (tabular)

Feature	Mean before	SD before	Mean after	SD after
days_since_start	31.43	16.71	-5.80e-17	1.000
stage	1.80	1.22	-1.95e-17	1.000
progress	36.11	23.52	+4.49e-15	1.000
pct_tasks_complete	0.36	0.22	-1.21e-16	1.000
days_since_last_activity	2.82	3.90	-5.20e-16	1.000
docs_pending	2.13	2.28	+3.91e-17	1.000

All 12 features standardise to mean ~0 and SD 1, which is what makes the logistic regression coefficients directly comparable as per-feature contributions.

5. Data splitting and leakage

Dataset	Split	Train	Test	Stratified	Max class drift	Train/test overlap
Documents	75/25	3,712	1,238	yes	0.0001	0
Messages	75/25	2,520	840	yes	0.0000	70
Journeys	75/25	4,500	1,500	yes	0.0002	n/a

- `random_state=42` on every split, so results are reproducible.
- Stratification keeps every class proportion within 0.0001 of the full dataset.
- **Leakage controls:** the vectoriser and the scaler are fitted inside an sklearn `Pipeline`, so they are re-fitted on the training fold alone during cross-validation and never see the test fold. Cross-validation uses the full dataset only through that Pipeline, never a pre-fitted transformer.
- **Documents:** 0 records overlap between folds. Clean.
- **Messages: 70 records overlap between folds (8.3% of the test set).** This is genuine leakage caused by duplicate short messages, and it inflates the intent and sentiment scores. Deduplicating before the split, or grouping identical strings into the same fold, is the required fix and is listed in future work.